Módulo 6: Banco de Dados (resumo teórico)

Resumo teórico em português para estudo. Cobre o escopo do módulo 6 do edital do TCE-GO (Técnico de Controle Externo, TI): modelagem de dados, SQL, álgebra relacional, transações e ACID, índices, procedures, triggers e views, PostgreSQL e Oracle, NoSQL, bancos vetoriais, replicação, backup, alta disponibilidade, segurança e governança. Os exemplos de SQL foram executados em PostgreSQL 18.4 e SQLite 3.46.1; Oracle, SQL Server, MySQL, MongoDB e Redis não foram executados (não testado).

1. Modelos de dados, SGBD e níveis de abstração

- Um banco de dados é uma coleção organizada de dados persistentes; o SGBD (sistema gerenciador de banco de dados) é o software que define, armazena, consulta e protege esses dados. Exemplos: PostgreSQL, Oracle Database, MySQL, SQL Server.
- Um SGBD oferece persistência, linguagem de consulta, controle de concorrência, recuperação de falhas, segurança e integridade. A independência de dados separa quem usa os dados de como eles são guardados.
- Níveis de modelagem, do mais abstrato ao mais concreto: modelo conceitual (entidades e relacionamentos, sem tecnologia, em geral o MER), modelo lógico (tabelas, colunas e chaves do modelo relacional, com normalização) e modelo físico (tipos, índices, partições e armazenamento do SGBD escolhido).
- Arquitetura de três esquemas (ANSI/SPARC): esquema externo (visões dos usuários), conceitual (o banco como um todo) e interno (armazenamento).
- Linguagens: DDL define estruturas, DML manipula linhas, DCL controla permissões, DQL consulta (o SELECT) e TCL controla transações (COMMIT, ROLLBACK, SAVEPOINT).
- Modelos de dados: relacional (tabelas), hierárquico e em rede (históricos), orientado a objetos e NoSQL (documentos, chave-valor, colunas, grafos).

2. Modelo Entidade-Relacionamento (MER)

- Entidade é algo do mundo real sobre o qual se guardam dados (Cliente, Processo). Atributo é uma propriedade da entidade. Relacionamento associa entidades.
- Tipos de atributo: simples (indivisível), composto (divisível em subpartes com significado, como endereço), multivalorado (vários valores, como telefones), derivado (calculado, como idade a partir do nascimento) e identificador (chave).
- Cardinalidade: 1:1, 1:N e N:N. Grau do relacionamento: unário ou auto relacionamento (a entidade com ela mesma), binário (duas entidades) e ternário (três).
- Participação: total (obrigatória) ou parcial (opcional).
- Entidade fraca: não tem identificador próprio suficiente e depende de uma entidade forte; sua chave é a chave da entidade forte mais um discriminador (Dependente de Funcionário).
- Entidade associativa: transforma um relacionamento em entidade, para que ele possa ter atributos ou se relacionar com outras entidades.
- Especialização e generalização (EER): especialização divide um supertipo em subtipos com atributos próprios (Cidadão em Pessoa Física e Pessoa Jurídica); generalização é o

caminho inverso. Pode ser total ou parcial, disjunta ou sobreposta.
- Notações: Chen (losango para relacionamentos), pé de galinha (crow's foot) e UML. O MER é o modelo conceitual por excelência.

3. Modelo relacional, chaves e integridade

- Relação (tabela) é um conjunto de tuplas (linhas) com os mesmos atributos (colunas); cada atributo tem um domínio. Em teoria não há tuplas duplicadas nem ordem; o SQL relaxa isso.
- Superchave identifica unicamente a linha; chave candidata é a superchave mínima; chave primária é a candidata escolhida; chave alternativa são as candidatas restantes; chave estrangeira referencia a chave de outra tabela; chave substituta (surrogate) é um identificador artificial, como um SERIAL.
- Integridade de entidade: a chave primária não é nula nem repetida. Integridade referencial: toda chave estrangeira aponta para uma linha existente ou é nula. Integridade de domínio: tipos, NOT NULL e CHECK.
- Ações referenciais: NO ACTION e RESTRICT (recusam), CASCADE (propaga), SET NULL e SET DEFAULT.
- Restrições em SQL: PRIMARY KEY, UNIQUE (aceita NULL, que não conta como repetição), NOT NULL, CHECK (só reprova quando a condição é falsa: com NULL ela é desconhecida e a linha passa) e FOREIGN KEY. Restrições podem ter nome: `CONSTRAINT nome UNIQUE (a, b)` torna único o par.
- Mapeamento MER para relacional: entidade vira tabela; atributo composto vira colunas; atributo multivalorado vira outra tabela; 1:N leva a chave do lado 1 como estrangeira ao lado N; N:N vira tabela associativa com duas chaves estrangeiras (chave composta); 1:1 usa a chave de um lado como chave estrangeira (ou a mesma chave nos dois); entidade fraca recebe a chave do dono como parte da chave.
- Tipos comuns: INTEGER, NUMERIC(p,s), VARCHAR(n), TEXT, DATE, TIMESTAMP, BOOLEAN. SERIAL e IDENTITY geram sequências. No Oracle: NUMBER, VARCHAR2, sequências e DUAL.

4. Álgebra relacional

- É a base teórica do SQL: operações sobre relações que devolvem relações.
- Seleção σ(condição) filtra linhas (WHERE). Projeção π(colunas) escolhe colunas e elimina duplicatas (SELECT DISTINCT). Renomeação ρ muda nomes (AS).
- Operações de conjunto: união ∪, interseção ∩ e diferença −, que exigem relações compatíveis (mesmas colunas e domínios). Produto cartesiano × combina todas as linhas com todas.
- Junção ⋈ é o produto seguido de seleção. Junção natural liga por atributos de mesmo nome; theta-join usa uma condição qualquer; equijoin usa igualdade. Junções externas (esquerda, direita e completa) preservam linhas sem par.
- Divisão ÷ responde "para todo" (clientes que compraram todos os produtos). Agregação γ agrupa e calcula (GROUP BY).
- Equivalências: σ e π podem ser empurradas para perto das tabelas para reduzir o volume antes da junção; é uma das bases da otimização. Operadores unários: σ, π e ρ;

binários: ∪, ∩, −, ×, ⋈ e ÷.
- Cálculo relacional: linguagem declarativa (o que obter, não como), base do SQL; a álgebra é procedural.

5. Normalização e dependências funcionais

- Normalizar é decompor tabelas para reduzir redundância e evitar anomalias de inserção, atualização e exclusão, sem perder informação.
- Dependência funcional X → Y: cada valor de X determina um único valor de Y. Axiomas de Armstrong: reflexividade, aumento e transitividade (daí saem união, decomposição e pseudotransitividade). O fecho de um conjunto de atributos X+ é tudo o que X determina; X é chave se X+ é a relação toda.
- 1FN: valores atômicos, sem grupos repetidos nem atributos multivalorados nas células.
- 2FN: está na 1FN e nenhum atributo não chave depende de apenas parte de uma chave composta (sem dependência parcial).
- 3FN: está na 2FN e nenhum atributo não chave depende de outro não chave (sem dependência transitiva). Regra de memória: cada atributo depende da chave, de toda a chave e nada além da chave.
- FNBC (Boyce-Codd): para toda dependência X → Y não trivial, X é superchave. É mais forte que a 3FN.
- 4FN trata dependências multivaloradas; 5FN, dependências de junção.
- Boa decomposição: sem perdas (a junção reconstrói a original) e, de preferência, preservando as dependências.
- Quando a prova apresenta dependências e pede a forma normal, ache as chaves pelo fecho e compare cada determinante com elas.

6. Desnormalização

- Desnormalizar é, de propósito, repetir ou reunir dados para evitar junções e acelerar leituras, em troca de mais espaço e do risco de anomalias de atualização.
- Técnicas: copiar colunas, tabelas resumo ou pré-calculadas, colunas derivadas, visões materializadas, fundir tabelas 1:1 e armazenar a hierarquia achatada.
- Faz sentido em cargas de leitura intensa, como relatórios, BI e data warehouse; costuma ser errado em sistemas transacionais (OLTP) com muita escrita.
- Em NoSQL de documentos, o modelo usual embute dados relacionados no mesmo documento: é uma desnormalização deliberada para evitar junções.
- Sempre se normaliza primeiro e só depois, se houver medição que justifique, se desnormaliza.

7. SQL: DDL, DML, DCL e DQL

- DDL: `CREATE`, `ALTER`, `DROP` e `TRUNCATE` (esvazia a tabela, sem WHERE). DML: `INSERT`, `UPDATE`, `DELETE` e `MERGE`. DQL: `SELECT`. DCL: `GRANT` e `REVOKE`. TCL: `COMMIT`, `ROLLBACK` e `SAVEPOINT`. Algumas fontes tratam o SELECT como parte da DML.
- `UPDATE tabela SET coluna = valor WHERE condição`: sem WHERE todas as linhas mudam. `DELETE FROM tabela WHERE condição` apaga linhas; `DROP TABLE` apaga a

estrutura; `TRUNCATE` apaga todas as linhas e costuma ser mais rápido.
- `INSERT INTO t (a, b) VALUES (1, 'x'), (2, 'y')` aceita várias linhas no PostgreSQL. `RETURNING *` devolve as linhas inseridas no PostgreSQL. `MERGE` insere ou atualiza conforme a correspondência (PostgreSQL 15 em diante e Oracle).
- `ALTER TABLE t ADD COLUMN c INT`, `DROP COLUMN`, `ADD CONSTRAINT`, `DROP CONSTRAINT nome`. `DROP TABLE IF EXISTS t` não dá erro se a tabela não existir.
- DCL: `GRANT SELECT ON t TO usuario` e `REVOKE`. No Oracle, privilégios de sistema também se concedem com GRANT (`GRANT CREATE SESSION, CREATE TABLE TO analista`); no PostgreSQL, os privilégios são de objeto e de papel (role).
- Dialetos: Oracle usa VARCHAR2, NUMBER, DUAL, `FETCH FIRST n ROWS ONLY` (e ROWNUM nos antigos); PostgreSQL usa SERIAL, `LIMIT`, `RETURNING`; SQL Server usa IDENTITY e `TOP`; SQLite tem tipagem dinâmica e não tem GRANT.

8. SQL: consultas, junções e subconsultas

- Ordem lógica de avaliação: FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT. Por isso um apelido definido no SELECT não vale no WHERE.
- `SELECT DISTINCT` elimina repetidas; `ORDER BY coluna DESC` ordena do maior para o menor; `LIKE 'anali%'` casa o início (`%` é qualquer sequência e `_` é um caractere); `IN (10, 20)` equivale a vários OR; `BETWEEN a AND b` é inclusivo.
- Junções: `INNER JOIN` devolve só os pares; `LEFT` e `RIGHT JOIN` preservam um lado; `FULL JOIN` preserva os dois; `CROSS JOIN` é o produto cartesiano. Esquecer a condição de junção produz o produto cartesiano (m × n linhas).
- Colunas de mesmo nome em duas tabelas exigem prefixo ou apelido (`f.depno`), senão o erro é "ambiguous".
- Subconsultas: escalar (devolve um valor), de lista (IN, NOT IN, ANY, ALL), de tabela (no FROM) e correlacionada (usa colunas da consulta externa, avaliada por linha). `EXISTS` testa se há alguma linha.
- Anti-junção (quem não tem correspondente): `NOT EXISTS`, `LEFT JOIN ... WHERE chave IS NULL` ou `NOT IN`. O `NOT IN` devolve vazio se a subconsulta tiver algum NULL, por isso `NOT EXISTS` é o mais seguro.
- Conjuntos: `UNION` (sem repetidas), `UNION ALL`, `INTERSECT` e `EXCEPT` (`MINUS` no Oracle).
- Funções de texto: `REPLACE(coluna, 'a', 'b')` troca sem alterar a tabela (só a exibição); no Oracle, `REGEXP_LIKE(coluna, padrão)` testa expressões regulares; aspas simples são texto e aspas duplas são identificadores.
- CTE: `WITH nome AS (SELECT ...)`. `WITH RECURSIVE` percorre hierarquias e grafos.

9. SQL: agregação, janelas e NULL

- Agregações: COUNT, SUM, AVG, MIN e MAX. `COUNT(*)` conta linhas; `COUNT(coluna)` ignora NULL; `AVG` também ignora NULL. `COUNT(DISTINCT c)` conta valores diferentes.
- `GROUP BY` agrupa; toda coluna não agregada do SELECT precisa estar no GROUP BY (o PostgreSQL recusa e o SQLite aceita). `WHERE` filtra linhas antes de agrupar; `HAVING` filtra grupos depois. Função de agregação não pode ficar no WHERE: use subconsulta ou HAVING.

- Máximo por grupo sem janela: `WHERE NOT EXISTS (outra linha do mesmo grupo com valor maior)`, que devolve os empates.
- Funções de janela: `RANK()`, `ROW_NUMBER()`, `LAG()`, `SUM(...) OVER (PARTITION BY ... ORDER BY ...)`, médias móveis. A janela não colapsa as linhas, ao contrário do GROUP BY.
- Lógica de três valores: com NULL, comparações dão desconhecido (UNKNOWN). `C = NULL` nunca é verdadeiro; use `IS NULL`. NOT desconhecido é desconhecido; falso AND desconhecido é falso; verdadeiro OR desconhecido é verdadeiro. WHERE e CASE WHEN só aceitam a linha quando a condição é verdadeira.
- `COALESCE(a, b)` devolve o primeiro não nulo; `NULLIF(a, b)` devolve NULL se forem iguais.
- Divisão inteira: no SQLite e no PostgreSQL, `7 / 2` entre inteiros dá 3; use `7 / 2.0` ou CAST para obter decimais. Em Oracle, `7 / 2` dá 3,5.
- `ROLLUP`, `CUBE` e `GROUPING SETS` geram subtotais (PostgreSQL, Oracle; SQLite 3.46 não tem).

10. Transações e propriedades ACID

- Transação é uma unidade lógica de trabalho: um conjunto de operações que deve ser executado por inteiro ou nenhuma.
- ACID: Atomicidade (tudo ou nada: em caso de erro, `ROLLBACK` desfaz tudo); Consistência (leva o banco de um estado válido a outro, respeitando as restrições); Isolamento (transações concorrentes não enxergam resultados parciais umas das outras); Durabilidade (depois do COMMIT, o resultado sobrevive a falhas).
- Comandos: `BEGIN` (ou `START TRANSACTION`), `COMMIT`, `ROLLBACK`, `SAVEPOINT nome` e `ROLLBACK TO nome` (desfaz só parte).
- Autocommit: cada comando é uma transação própria. No PostgreSQL, uma transação com erro fica "abortada" até o ROLLBACK.
- Durabilidade e recuperação: o log (WAL no PostgreSQL; redo log no Oracle) grava a mudança antes do dado ir para o disco (write-ahead logging). Após falha, refaz (redo) o que foi confirmado e desfaz (undo) o que não foi.
- Checkpoint: ponto em que as páginas sujas são gravadas no disco, encurtando a recuperação.
- DDL no PostgreSQL é transacional (um CREATE TABLE pode ser desfeito); no Oracle, o DDL faz commit implícito.
- Em sistemas distribuídos, o commit em duas fases (2PC) coordena várias bases; microsserviços preferem sagas.

11. Concorrência: isolamento, bloqueios e deadlock

- Anomalias: leitura suja (dirty read: lê dado não confirmado), leitura não repetível (non-repeatable read: a mesma linha muda entre duas leituras), leitura fantasma (phantom read: aparecem novas linhas que satisfazem o filtro) e atualização perdida (lost update).
- Níveis do padrão SQL: READ UNCOMMITTED (permite as três), READ COMMITTED (impede a leitura suja), REPEATABLE READ (impede a não repetível) e SERIALIZABLE

(impede todas). O padrão do PostgreSQL e do Oracle é READ COMMITTED.
- PostgreSQL: READ UNCOMMITTED se comporta como READ COMMITTED; REPEATABLE READ é isolamento por instantâneo (snapshot) e também impede fantasmas; SERIALIZABLE usa SSI e pode falhar com erro de serialização, que deve ser repetido. Oracle oferece READ COMMITTED, SERIALIZABLE e somente leitura.
- MVCC (controle de concorrência multiversão): cada transação lê uma versão consistente; leitores não bloqueiam escritores e vice-versa. No PostgreSQL as versões antigas ficam na própria tabela e o VACUUM as limpa; no Oracle, ficam no segmento de undo.
- Bloqueios: compartilhado (leitura) e exclusivo (escrita); por linha, página ou tabela. `SELECT ... FOR UPDATE` bloqueia as linhas lidas e evita a atualização perdida. `LOCK TABLE` bloqueia a tabela.
- Protocolo de bloqueio em duas fases (2PL): fase de aquisição e fase de liberação; garante serializabilidade.
- Deadlock: duas transações esperam uma pela outra. O SGBD detecta (grafo de espera) e aborta uma delas ("deadlock detected"). Prevenção: acessar recursos sempre na mesma ordem e usar transações curtas.
- Controle otimista (versão ou carimbo de tempo) assume poucos conflitos e valida no final; o pessimista usa bloqueios.

12. Índices e otimização de consultas

- Índice é uma estrutura auxiliar que acelera buscas à custa de espaço e de escritas mais lentas (todo INSERT, UPDATE e DELETE atualiza os índices).
- B-tree (Árvore B ou B+): árvore balanceada, com todas as folhas no mesmo nível; busca, inserção e remoção custam O(log n). Atende igualdade, faixa e ordenação. É o tipo padrão.
- Outros: hash (só igualdade), bitmap (poucos valores distintos, comum em DW e no Oracle), GIN (valores compostos, como JSONB, arrays e texto), GiST, BRIN (tabelas enormes ordenadas fisicamente) e índices vetoriais HNSW e IVF.
- Primário (clustered): define a ordem física dos dados (SQL Server e InnoDB); secundário (não agrupado): aponta para as linhas. Índice composto vale pelo prefixo mais à esquerda. Índice de cobertura (covering) atende a consulta sem ler a tabela. Índice parcial cobre só parte das linhas. UNIQUE garante unicidade.
- Sargabilidade: uma condição é sargável se o índice pode ser usado. Aplicar função à coluna (`WHERE UPPER(nome) = 'ANA'`, `WHERE YEAR(data) = 2025`) ou usar `LIKE '%x'` desativa o índice; use índice funcional ou reescreva a condição.
- Seletividade: o índice vale para filtros que devolvem poucas linhas; em colunas com poucos valores, o otimizador prefere varrer a tabela.
- Otimizador baseado em custo: usa estatísticas (`ANALYZE`) para escolher o plano. Algoritmos de junção: laço aninhado (nested loop), hash join e merge join.
- Plano de consulta: sequência de operações (varredura sequencial, por índice, junção, ordenação) com estimativa de custo. `EXPLAIN` mostra o plano; `EXPLAIN ANALYZE` executa e mostra também o tempo real. No Oracle, `EXPLAIN PLAN` e `DBMS_XPLAN`.
- Dicas práticas: indexar chaves estrangeiras e colunas de filtro e ordenação; evitar `SELECT *`; filtrar cedo; evitar funções na coluna; manter estatísticas atualizadas;

particionar tabelas muito grandes.

13. Views, procedures, functions e triggers

- View é uma consulta nomeada, uma tabela virtual: simplifica consultas, esconde colunas e linhas (segurança) e dá independência lógica. View materializada guarda o resultado e precisa ser atualizada (`REFRESH MATERIALIZED VIEW`).
- Views simples podem aceitar INSERT, UPDATE e DELETE; as com junção, agregação ou DISTINCT em geral não são atualizáveis. `WITH CHECK OPTION` impede alterações que tirem a linha da view.
- Function (função) devolve um valor e pode ser usada em consultas. Procedure (procedimento) é chamada com `CALL`/`EXECUTE` e pode controlar transações (PostgreSQL 11 em diante). Oracle usa PL/SQL; PostgreSQL usa PL/pgSQL; SQL Server usa T-SQL.
- Trigger (gatilho) executa automaticamente em INSERT, UPDATE ou DELETE. Momento: BEFORE, AFTER ou INSTEAD OF. Escopo: FOR EACH ROW (por linha) ou FOR EACH STATEMENT. Usa as linhas OLD e NEW.
- Usos de triggers: auditoria, validações complexas, derivar valores. Cuidados: lógica escondida, efeito cascata e custo em cargas grandes. Restrições declarativas (CHECK, FK) são preferíveis quando bastam.
- Cursor: percorre o resultado linha a linha dentro de procedimentos; é lento perto de operações em conjunto.
- Vantagens das procedures: menos tráfego de rede, reutilização e segurança (GRANT EXECUTE sem acesso às tabelas). Desvantagens: lógica de negócio no banco e dependência do fornecedor.

14. PostgreSQL e Oracle Database: administração e otimização

- PostgreSQL: SGBD objeto-relacional de código aberto. Processos, memória compartilhada (`shared_buffers`), WAL, MVCC e VACUUM/autovacuum, que remove versões mortas e atualiza estatísticas. Tabelas, esquemas (schemas), tablespaces, papéis (roles) e extensões (pgvector, PostGIS). Tipos ricos: arrays, JSON e JSONB, hstore, range e geométricos.
- Configuração: `postgresql.conf` (parâmetros) e `pg_hba.conf` (quem pode se conectar, de onde e como se autentica). Ferramentas: `psql`, `pg_dump`, `pg_restore`, `pg_basebackup`, `pg_ctl`, `pg_stat_statements` e `EXPLAIN (ANALYZE, BUFFERS)`.
- Declaração de array: `teste integer[][]` indica duas dimensões; o PostgreSQL não impõe tamanho nem número de dimensões.
- Oracle Database: instância (processos e memória: SGA e PGA) e banco (arquivos de dados, redo logs, arquivos de controle). Tablespaces, segmentos, undo, redo e tabela `DUAL`. Linguagem PL/SQL. Backup com RMAN; alta disponibilidade com RAC, Data Guard e Flashback; diagnóstico com AWR, ADDM e `EXPLAIN PLAN`.
- Oracle: isolamento por undo, sequências, `VARCHAR2` e `NUMBER`, `ROWNUM` e `FETCH FIRST`, hints no SQL e `SELECT UNIQUE` (sinônimo de DISTINCT, só no Oracle).

- Otimização no PostgreSQL: índices adequados, `ANALYZE`, ajuste de memória, reescrever consultas, particionamento e pool de conexões. No Oracle: estatísticas do otimizador, plano de execução, índices (B-tree e bitmap), particionamento e hints.
- MySQL: o mecanismo InnoDB é o transacional (ACID, chaves estrangeiras, bloqueio por linha); MyISAM, MEMORY (HEAP) e MERGE não são transacionais. Não testado: o laboratório não tem MySQL nem Oracle.

15. Bancos NoSQL

- NoSQL ("not only SQL"): modelos que abrem mão de parte do modelo relacional para ganhar escala horizontal, flexibilidade de esquema e desempenho em cenários específicos.
- Documentos (MongoDB, CouchDB): documentos JSON ou BSON em coleções; esquema flexível; consultas por campos e agregações. Bom para catálogos, perfis e conteúdo.
- Chave-valor (Redis, DynamoDB, Memcached): acesso por chave, latência baixíssima. O Redis guarda em memória, com estruturas (strings, hashes, listas, conjuntos, conjuntos ordenados), expiração (TTL), persistência (RDB e AOF) e pub/sub. Usado para cache, sessões e filas.
- Famílias de colunas ou wide-column (Cassandra, HBase): linhas com chave de partição e colunas agrupadas em famílias; grava com muita velocidade e escala linearmente; consultas planejadas pelas chaves.
- Grafos (Neo4j): nós, arestas e propriedades; linguagem Cypher; ideal para redes, recomendações e detecção de fraude, em que as relações importam mais que as linhas.
- Teorema CAP: em um sistema distribuído com partição de rede, só se garante C (consistência: toda leitura vê a escrita mais recente) ou A (disponibilidade: todo nó não falho responde); P (tolerância a partição) é inevitável. CP privilegia a consistência; AP, a disponibilidade.
- BASE: Basically Available, Soft state, Eventual consistency, em contraste com ACID. Consistência eventual: as réplicas convergem com o tempo.
- Sharding: particiona os dados horizontalmente entre nós (por faixa ou por hash de uma chave) para distribuir carga e armazenamento; uma má chave cria "hot spots". Replicação copia os dados para tolerância a falhas e leitura.
- Quando escolher: dados relacionais, transações e consultas ad hoc pedem SQL; escala massiva, esquema variável ou acesso simples por chave pedem NoSQL. É comum usar os dois (persistência poliglota). Modelagem em NoSQL parte das consultas, e não das entidades.

16. Bancos de dados vetoriais e embeddings

- Embedding é um vetor de números (centenas ou milhares de dimensões) produzido por um modelo, que representa o significado de um texto, imagem ou áudio: conteúdos parecidos geram vetores próximos.
- Banco vetorial armazena embeddings e faz busca por similaridade (vizinhos mais próximos). Métricas: distância euclidiana (L2), similaridade do cosseno e produto escalar.

- Busca exata (kNN) compara com todos os vetores. Busca aproximada (ANN) usa índices como HNSW (grafo hierárquico) e IVF (agrupamento por centroides) e troca um pouco de precisão (recall) por muita velocidade.
- Produtos: bancos dedicados (Pinecone, Milvus, Qdrant, Weaviate, Chroma) e extensões de bancos tradicionais (pgvector no PostgreSQL), que permitem combinar vetor e filtros SQL.
- Aplicações com IA: RAG (geração aumentada por recuperação, em que se buscam trechos relevantes antes de o modelo responder), busca semântica, recomendação e detecção de duplicatas.
- Fluxo típico: quebrar documentos em trechos, gerar embeddings, gravar com metadados, converter a pergunta em embedding, buscar os vizinhos e montar o contexto.
- Pontos de atenção: a dimensão do vetor depende do modelo e deve ser a mesma na gravação e na consulta; atualizar o modelo exige recalcular os vetores; governança e segurança dos dados continuam valendo.

17. Replicação, backup, recuperação e alta disponibilidade

- Replicação mantém cópias dos dados em outros servidores. Primário e réplica (um escreve, os outros leem); multi-mestre (vários escrevem, com resolução de conflitos). Síncrona (o commit espera a réplica: sem perda, mais latência) ou assíncrona (mais rápida, com risco de perder as últimas transações).
- PostgreSQL: replicação física por streaming de WAL (réplica idêntica, em espera ou somente leitura, com promoção em caso de falha) e replicação lógica (publicação e assinatura por tabela). Oracle: Data Guard e RAC.
- Backup lógico (`pg_dump`, exporta SQL ou formato próprio; portável, restaura com `pg_restore` ou `psql`) e físico (cópia dos arquivos, `pg_basebackup`, RMAN). Completo, incremental (desde o último backup de qualquer tipo) e diferencial (desde o último completo). A quente (com o banco no ar) ou a frio.
- Recuperação pontual (PITR): backup base mais arquivamento contínuo de WAL permitem restaurar até um instante exato, por exemplo, antes de um DELETE errado.
- Objetivos: RPO (quanta perda de dados se aceita, medida em tempo) e RTO (quanto tempo se aceita ficar fora). Regra 3-2-1: três cópias, em duas mídias, uma fora do local. Backup que nunca foi restaurado não é backup: teste a restauração.
- Alta disponibilidade: elimina pontos únicos de falha com redundância, failover automático, quorum e monitoramento; cuidado com o split-brain (dois nós achando que são o primário). Ferramentas: Patroni, repmgr, Pacemaker, balanceadores. Disponibilidade de 99,9% são cerca de 8h46 de parada por ano.
- Replicação não substitui backup: um comando destrutivo é replicado. Recuperação de desastres inclui site remoto e plano testado.

18. Segurança e governança de dados

- Tríade CID: confidencialidade, integridade e disponibilidade. Controles: identificação e autenticação (senha forte, MFA, certificados), autorização (privilégio mínimo, papéis), auditoria e criptografia.

- Autorização em SQL: `GRANT` e `REVOKE` por objeto e por coluna; papéis agrupam privilégios; views escondem colunas e linhas. Segurança em nível de linha (RLS) filtra as linhas por usuário.
- Criptografia em repouso (discos, colunas, backups) e em trânsito (TLS). Mascaramento e anonimização protegem dados em ambientes de teste. Gestão de chaves separada dos dados.
- SQL Injection: o texto do usuário é interpretado como código. Defesa: consultas parametrizadas (prepared statements), validação de entrada e privilégio mínimo da conta da aplicação; nunca concatenar texto na consulta.
- Auditoria e rastreabilidade: registrar quem acessou ou mudou o quê, com logs protegidos contra alteração.
- Governança de dados: políticas, papéis (dono, curador ou steward, custodiante), catálogo e dicionário de dados, qualidade, linhagem (de onde vem e para onde vai), ciclo de vida, retenção e classificação por sensibilidade. DAMA-DMBOK é a referência.
- LGPD (Lei 13.709/2018): dados pessoais exigem base legal e finalidade; direitos do titular; encarregado (DPO); relatório de impacto; incidente deve ser comunicado.
- Qualidade de dados: exatidão, completude, consistência, unicidade, atualidade e validade.

19. Data warehouse, OLAP e modelagem dimensional

- OLTP: muitas transações curtas, normalizado, atual. OLAP: consultas analíticas pesadas, sobre histórico, dimensional. Data warehouse (DW): repositório integrado, histórico e não volátil, orientado por assunto. Data mart: subconjunto do DW por área. Data lake: dados brutos de formatos variados, com esquema na leitura. Lakehouse: junta o lake com transações ACID e esquema (Delta Lake, Iceberg).
- ETL (extrair, transformar, carregar) prepara os dados antes de entrarem no DW; ELT carrega primeiro e transforma no destino.
- Modelagem dimensional (Kimball): tabela fato (medidas numéricas e chaves das dimensões) e dimensões (contexto: tempo, produto, loja, cliente). Esquema estrela: dimensões desnormalizadas, poucas junções. Esquema floco de neve: dimensões normalizadas em subdimensões, menos espaço e mais junções.
- Medidas: aditiva (soma em todas as dimensões: valor da venda), semiaditiva (soma só em algumas: saldo de conta não soma no tempo) e não aditiva (razões e percentuais, que não se somam).
- Chave substituta (surrogate key) nas dimensões. Dimensão lentamente mutável (SCD): tipo 1 sobrescreve; tipo 2 cria nova linha com vigência (guarda o histórico); tipo 3 guarda o valor anterior em outra coluna.
- Operações OLAP: roll-up e drill-down (mais ou menos detalhe), slice (corte em uma dimensão), dice (corte em várias) e pivot (rotação). Armazenamentos: ROLAP, MOLAP e HOLAP. SQL: `ROLLUP`, `CUBE` e funções de janela.
- Pontos de prova: o DW é voltado a leitura em grande volume, com poucas escritas concorrentes, e não sofre de deadlocks como o OLTP; a tabela fato guarda as medidas e o contexto vem das dimensões.